

Strategic Plan for an Advanced MCP-based AI Co-Developer

Background and Objectives

We propose an enhanced Machine Control Protocol (MCP) architecture that transforms a custom MCP system into an **intelligent AI co-developer**. The goal is to support real-time code co-development, architectural reasoning, automated QA, long-term knowledge retention, and intelligent predictions. Key capabilities include: specialized developer agents, multi-agent collaboration, continuous learning from trusted code repositories, and seamless integration with developer tools (e.g. GitHub CLI, VS Code APIs). The system will run on both **cloud** and **on-premise** infrastructure, leveraging containers or serverless environments for deployment. It will use autonomous AI agents that invoke custom tools via MCP, and will continually retrain on validated data pipelines for ongoing improvement.

Building on examples from Octopus Deploy's AI MCP and GitHub Copilot's custom agent framework, our strategy emphasizes support for software engineering tasks (code search, generation, review), system architecture design, and QA workflows. Drawing on DeepMind's breakthroughs (AlphaCode, AlphaGeometry, AlphaDev) and Google's Gemini 2.5 reasoning techniques, we will implement advanced reasoning and self-improvement loops. This document outlines the architecture, agent patterns, pipelines, and training loops needed to realize these goals.

Use Cases and Inspiration

- **Octopus Deploy MCP use cases:** Octopus's AI MCP demonstrates how agents can **reason about deployments** and configuration. For example, it can answer which version is running on a customer's environment and diagnose deployment failures ¹ ². These use cases highlight capabilities like change management (identifying deployed versions, bug impacts) and troubleshooting (analyzing failed deployments to suggest fixes). We will incorporate similar features – e.g., agents that can query environment state, trace errors, and recommend fixes via logs and telemetry.
- **GitHub Copilot custom agents:** GitHub's customizable agents allow defining specialized “sub-agents” with distinct prompts and tools. The Copilot changelog notes that developers can create domain-specific agents (e.g. a React frontend agent) that use **MCP servers and GitHub CLI tools** ³ ⁴. For instance, an agent might automate GitHub tasks (creating PRs, managing issues) using a CLI MCP tool. We will leverage this approach by building custom agents that invoke our own MCP tools for code repositories, issue trackers, documentation, etc., while following organization-specific guidelines. This enables **fine-grained control** of agent behavior and tool use ³.
- **VS Code extension and APIs:** VS Code supports **agent mode** and AI tools integration. Its AI extensibility guide emphasizes embedding **domain-specific tools and knowledge** into AI features

⁵ . For example, we can create a VS Code extension that exposes project-specific data (code snippets, architecture docs) to the AI via MCP, improving context-aware suggestions. We will use the VS Code extension APIs to integrate our MCP clients, so developers can invoke AI assistants directly in the IDE, with capabilities like custom tool windows or chat interfaces. Copilot's documentation also shows that built-in MCP servers already provide tools for repo and file operations ⁶ ; we will similarly register custom tools (based on GitHub CLI, local services, etc.) so agents can read/write code, query data, and orchestrate workflows.

- **Octocode & GitHub CLI:** The *Octocode* project by Wix demonstrated the power of using the **GitHub CLI** as a backbone for code-centric agents ⁷ . Instead of low-level API calls, they leveraged CLI commands (handling authentication and limits) to index and retrieve code and docs. We will adopt a similar strategy: build MCP tools that wrap GitHub CLI commands (e.g. `gh search code`, `gh pr create`) and other developer CLIs. This simplifies secure access to repository data and enables pipeline automation (like auto-generating release notes or verifying merge readiness) as part of agents' workflows ⁷ .

System Architecture

Our design follows a **multi-agent orchestration** model (see Figure below). At the top, a *coordinator agent* (orchestrator) directs the flow, spawning specialized subagents for distinct tasks ⁸ ⁹ . For example, one subagent might focus on code generation, another on architecture analysis, and another on running QA tests. Each agent runs as a containerized service (e.g. on Kubernetes or serverless Cloud Run ¹⁰), and communicates with others via the **Agent-to-Agent (A2A) protocol** for interoperability ⁸ . All agents use the MCP (Model Context Protocol) to invoke external tools: e.g. MCP clients in each agent send requests to MCP servers that manage access to databases, file systems, or the GitHub CLI ¹¹ .

Figure: Example multi-agent architecture (lead agent orchestrates parallel subagents). Each subagent has its own model, memory, and toolset (e.g. code search, QA tools). This orchestrator-worker pattern allows parallel reasoning and specialization ⁹ ⁸ .

In this architecture:

- The **Frontend** (e.g. VS Code or web UI) sends user prompts or events to the coordinator agent.
- The **Coordinator/Lead Agent** parses intents (e.g. "refactor this module", "debug build failure"), formulates a high-level plan using an LLM, and dispatches subagents with specific instructions ⁹ ⁸ .
- **Subagents** are specialized: one might do static analysis, another generate code, another run tests. They operate **in parallel** or in pipelined sequences (see Agent Collaboration Patterns below). Subagents communicate results back to the lead agent via shared state or the A2A protocol ⁸ .
- **MCP Tool Servers** expose functionalities (e.g. Git operations, database queries, cloud APIs). Agents call these through MCP. For example, a "GitHub MCP tool" can list repos or create PRs ⁶ , while a "Deployment MCP tool" can query CI/CD pipelines. By centralizing tools via MCP, agents remain modular and secure ¹¹ .
- **Memory and Long-Term Storage:** Agents maintain contextual memory (see below) and a model registry or knowledge base (e.g. vector stores for technical docs). This memory is persisted between sessions for long-term learning.

This cloud-agnostic reference architecture allows deployment on cloud or on-premise. Agents can run on Kubernetes clusters or serverless compute; models can be hosted on cloud ML services (e.g. Vertex AI ¹⁰) or on local GPU clusters. A **Model Development Pipeline** (described later) handles training and updates,

with versioning via a model registry. The entire system is orchestrated to support **continuous integration/continuous deployment (CI/CD)** of both code and models (see Training Pipeline section).

Multi-Agent Collaboration Patterns

The system uses proven **multi-agent orchestration patterns** to coordinate tasks. As in Azure’s architecture guide, breaking complex problems into specialized agents yields *specialization*, *scalability*, and *maintainability* ¹². Below are key patterns:

Pattern	Description	Use Case Example
Sequential	Agents form a predefined pipeline (A → B → C). Each stage refines the output of the previous agent.	Code-generation workflow: one agent formats specs, next writes draft code, next reviews/refines it ¹³ .
Concurrent	Multiple agents work in parallel on the same task or subproblems. Their outputs are combined or compared.	Running diverse analyses at once (e.g. security scan, QA testing, performance profiling) to accelerate feedback ¹⁴ ¹⁵ .
Aggregator	(A type of concurrent pattern) Independent agents each produce a result; an aggregator agent merges them.	Generating multiple code variants (like AlphaCode sampling) and selecting the best via testing ¹⁶ .
Loop / Iterative Refinement	Agents iteratively propose solutions and critique them (a “maker-checker” loop).	One agent writes code, another reviews it, then prompts a rewrite; repeat until quality criteria are met (akin to group chat consensus) ¹⁷ ¹⁸ .
Handoff (Router)	Dynamic routing: one agent analyzes a task and forwards it to the most suitable agent on-the-fly.	Initial parsing of a user request and dispatching to an expert agent (e.g. frontend vs backend specialist) based on content.

These patterns mirror those in Anthropic’s multi-agent research system and Azure’s AI design patterns. For example, our coordinator can use the **sequential** pattern for structured workflows, or **concurrent/aggregator** when exploring many possibilities in parallel ⁹ ¹⁵. Agents may engage in **group-chat style collaboration** when brainstorming solutions: the lead agent moderates a chat among specialized agents that debate design trade-offs until consensus (group chat / maker-checker pattern) ¹⁹ ²⁰.

These multi-agent strategies enable ensemble reasoning: e.g., an architecture suggestion can be evaluated by independent “peer” agents that provide alternative views, and a final decision is made by majority or confidence voting ¹⁵ ²⁰. In practice, parallelizing agents greatly increases throughput for complex tasks. As Anthropic reports, distributing work across subagents with separate context windows can outperform a single agent by >90% on research benchmarks ²¹ ²². We will leverage this by spawning subagents (using techniques like Microsoft’s AutoGen or Anthropic’s A2A) to explore tasks in breadth (multiple angles) and depth (iterative refinement).

Tools Integration and Agent Extensions

To maximize developer productivity, we will tightly integrate with existing dev toolchains:

- **MCP Tools:** We will provide a suite of **custom MCP tools** for developer operations. This includes tools for GitHub (via CLI), local file system queries, IDE APIs, CI/CD status checks, cloud resource inspectors, static analysis, etc. For example, an agent could call a `listReleases()` tool to see recent versions (similar to Copilot's GitHub MCP offering repository tools ⁶), or a `runTests()` tool to execute unit tests. The MCP abstraction ensures agents use these tools uniformly. We take inspiration from Copilot's model by hosting the MCP server (the extension's backend) either locally or in the cloud, so agents can seamlessly invoke commands without exposing credentials ⁶.
- **GitHub CLI & Pipelines:** As in Octocode, we will **leverage GitHub CLI** within agents. For example, an agent needing code samples could call `gh repo clone`, `gh pr view`, or `gh search code`. Pipelines can be scripted (GitHub Actions or a custom process) to feed new data into the learning loop: e.g., on each merge to `main`, a pipeline triggers the AI system to retrain on the updated code and docs. Using CLI tools simplifies auth and standardizes queries ⁷.
- **VS Code Extension & APIs:** A dedicated VS Code extension will host our AI chat and agent features. Using VS Code's AI API, we'll register our MCP clients as **Language Model Tools** and **Chat Participants** ⁵. This allows agents to, for instance, open files, search symbols, or use debuggers programmatically. The extension can also embed the AI personality: if the custom MCP has a built-in developer-friendly persona, the VS Code UI (chat pane or inline suggestions) will reflect that tone and reasoning depth when presenting answers.
- **Continuous QA Agents:** We will integrate automated QA as an agent role. For example, after code generation, a "QA agent" could run unit tests and static analyzers, then report issues back to the developer agent. This mimics AlphaCode's sampling-and-testing loop ¹⁶: the code generator agent produces variants, and the QA agent filters out incorrect ones by executing tests. Failures trigger a re-generation with hints. Over time, we can incorporate reinforcement learning (like RAGEN) where agents learn which prompts or code patterns yield fewer errors.

Training, Memory, and Continuous Learning

A robust training pipeline is key. We envision a CI/CD-style **ML pipeline** (see Figure below) that ingests trusted data, trains or fine-tunes models, evaluates them, and deploys updates:

Figure: ML lifecycle pipeline including data collection, preprocessing, model training/tuning, deployment, and monitoring (adapted from AWS Well-Architected ML lens ²³). Data from code repos and documentation is continuously ingested. A feature store/registry ensures consistent inputs; a model registry tracks versions ²³. After each code merge or validated data update, automated jobs retrain/fine-tune the model (e.g. a Gemini-based LLM) on recent code and issues. The training phase uses curated datasets (inspired by AlphaCode's competitive programming data ¹⁶ and AlphaGeometry's synthetic theorem generation ²⁴) and rigorous evaluation (unit tests, benchmarks). Inference-ready models are deployed (via containers or cloud services) for agents to use. Monitoring detects drift or quality issues, feeding back for retraining ²³.

Key aspects:

- **Synthetic and Fine-tuned Data:** We will generate specialized training data (e.g. code refactoring examples, geometry reasoning steps) as DeepMind did ²⁴. For instance, combining neural LMs with symbolic rules (à la AlphaGeometry) can create diverse code transformation scenarios without manual labels. Our pipeline will collect real developer workflows via GitHub CLI and use unit tests as implicit validation (similar to AlphaCode's test-validated dataset ¹⁶).
- **Reinforcement Learning:** Techniques like AlphaDev show the value of RL for optimizing code. We plan to incorporate RL-based agents that experiment (e.g. modifying algorithms) to maximize defined rewards (performance, correctness) ²⁵. For example, an "optimizer agent" could propose low-level code improvements (perhaps in assembly or optimized Python) and receive feedback from benchmarks, gradually discovering faster routines as AlphaDev did for sorting.
- **Deep Reasoning and Chain-of-Thought:** Models will be trained or fine-tuned to explicitly use **chain-of-thought** reasoning (as in Gemini 2.5) to solve complex engineering tasks ²⁶. This means the model will learn to "think aloud" internally (via special tokens) to improve logical accuracy. Training will incorporate prompts that encourage step-by-step solutions, verified by code execution or formal checks.

By continuously learning from the *validated ground truth* (approved code merges, documented architectures, resolved issues), the system refines its knowledge. For trust, each learning cycle only uses data from *trusted sources* (official repos, peer-reviewed patterns). The GitHub CLI and VS Code integration allow automatic pipeline triggers: e.g. a pipeline triggers on each PR merge, runs model retraining, and posts a report in the repo.

Long-Term Memory and Context Management

To support **long-term memory**, agents will store important information beyond a single conversation. For example, when tackling a large codebase, a lead agent can save its overall plan and key facts to memory (much like Anthropic's LeadResearcher did ²⁷). When token limits are reached, these memory entries are reloaded to preserve context (Gemini 2.5's 1M-token window helps, but persistent memory ensures no knowledge is lost ²⁸). We will use a combination of:

- **Vector Embeddings:** Store semantic embeddings of code comments, docs, Q&A pairs so agents can retrieve relevant knowledge quickly.
- **SQL/NoSQL Stores:** For structured info (e.g. architecture diagrams metadata, APIs), agents can query a knowledge base.
- **Conversational Memory:** Key conversation exchanges (e.g. "Final meeting decisions", "established code style rules") are logged so agents "remember" project conventions.

This memory mechanism turns the AI into a long-term collaborator that **learns a codebase over time**, improving with each interaction (like AlphaFold learning biology patterns, but here learning project patterns). Periodic summary agents can compress conversation histories into concise notes, which are then indexed. Memory access will be guarded by MCP so that agents retrieve only relevant, sanitized information.

Continuous Quality Assurance

Quality is enforced through automated testing and human-in-the-loop checks:

- **Automated Testing Agents:** Before presenting code to users, dedicated QA agents run static analysis, linters, and unit tests. Failing cases are reported as feedback, prompting code regeneration (similar to how

AlphaCode filters outputs with tests ¹⁶). Over time, this feedback loop trains the model to produce higher-quality code.

- **Human Oversight:** For critical tasks (e.g. code release), a human reviewer agent or process will sign off. Agents support human review by summarizing changes and highlighting risk areas. The Google Cloud example even includes a human-in-the-loop path for oversight ²⁹ .

- **Audit Trail:** All agent actions (code changes, tool calls) are logged. This ensures compliance and traceability, fulfilling audits as Octopus's use cases suggest. For instance, an "audit agent" can answer queries like "Who approved deployment X?" by searching logs.

Deployment Infrastructure

The system will support **hybrid deployment**. We can deploy core services (orchestration, agent runtimes, model servers) in the cloud for scalability, while sensitive components (private repos, PII data) remain on-premise. Both cloud (e.g. GKE/Cloud Run) and on-prem Kubernetes clusters are viable runtimes ¹⁰ . Models can be served via cloud ML platforms (Vertex AI or private inference servers).

Key considerations:

- **Scalability:** Use Kubernetes or serverless to scale agents elastically. The Google architecture shows agents as Cloud Run services or GKE pods ¹⁰ . We will do likewise, ensuring multiple instances of heavy agents (e.g. an inference model) can run concurrently.

- **Security & Model Armor:** Employ input sanitization layers (like Vertex AI Model Armor ³⁰) to prevent prompt injection. ACLs and environment isolation will secure the MCP tools.

- **CI/CD for Models:** Similar to code CI, model training and deployment will be automated (triggers on new data or policy changes). A **model registry** and container registry will version models, enabling rollback.

- **Offline/On-Prem Use:** For completely air-gapped environments, agents and tools will run locally with local data stores. The MCP protocol works over HTTP, so on-prem MCP servers can serve local tools without internet.

Advanced AI Capabilities

In building the AI brain, we leverage DeepMind and Google AI advances:

- **Gemini 2.5 (Deep Reasoning):** We will base our core LLM on a Gemini-class model or equivalent, which is "a thinking model" with built-in chain-of-thought reasoning ²⁶ . Gemini 2.5 excels at complex tasks (math, coding) and natively integrates tools (function calling) ³¹ ²⁶ . By using or fine-tuning such a model, agents can formulate and evaluate multiple reasoning steps internally, improving solution accuracy. Its 1M token context window ²⁸ supports long documents. We will also train the model to use its "tool integration" features to call our MCP tools dynamically during conversation ³¹ .

- **AlphaGeometry (Neuro-Symbolic Reasoning):** To enhance architectural reasoning, we adopt a neuro-symbolic approach. For example, when verifying software designs, an agent might use a symbolic logic engine in tandem with an LLM (much like AlphaGeometry's LM + theorem prover ³²). This allows rigorous checks (e.g. proving correctness of dependency graphs). We will also generate synthetic data for reasoning tasks (AlphaGeometry generated 100M synthetic problems ²⁴) – for instance, creating templated code problems to train the model on edge-case handling.

- **AlphaDev (Algorithm Optimization):** We incorporate RL-based optimization for critical code paths. Inspired by AlphaDev's assembly-level search ²⁵ , an "optimization agent" can automatically tweak algorithms (sorting routines, data structures) and test them for performance. Successful improvements can be submitted back to the repository (e.g. as PRs). This agent effectively performs low-level code evolution to

make applications faster and more efficient, one algorithm at a time ²⁵ .

- **AlphaCode (Code Generation & Filtering):** Our code synthesis strategy will mimic AlphaCode's generate-and-filter pipeline ¹⁶ . The code-generation agent will produce many solutions for a given problem, then a filtering agent (running test suites, style checks) will narrow to the best ones. We will fine-tune on large corpora of public code (similar to AlphaCode's pretraining) and competitive programming datasets ¹⁶ to improve creativity and correctness. Test-driven development is built into the loop.

- **Ensemble and Self-Improvement:** We may maintain an **ensemble of models** (different sizes or specialties) and an agent that selects the best model for a task (mirroring AlphaDev's model selection with RL). Additionally, as Gemini can revise its own prompts, we enable agents to refine their internal prompts (extended thinking modes ³³) or even auto-improve their tool descriptions ³⁴ , as demonstrated by Anthropic's tool-testing agent.

Integration Pipelines and Automation

Automated pipelines tie everything together. Example components include:

- **Code Ingestion:** A nightly job uses the GitHub CLI to pull new code, documentation, and issue histories. These are indexed for retrieval (e.g. in a vector DB for agent queries).

- **Validation and Training:** When significant changes occur (new release, large refactor), a pipeline runs the full test suite and benchmarks. Test results are stored. The AI training pipeline then (re)trains models on the updated repo snapshot (with augmented prompts that include test expectations).

- **Deployment:** Updated AI models are deployed as MCP servers or inference APIs. The VS Code extension or agent host is updated (via CI/CD) so developers always interact with the latest AI.

- **Monitoring and Feedback:** The system logs agent suggestions and outcomes. Metrics (e.g. suggestion acceptance rate, test pass improvements) feed into a monitoring dashboard. A feedback agent reviews logs to spot biases or errors, triggering retraining if needed.

This continuous loop ensures the MCP "learns" from each validation cycle, akin to ML best-practices ²³ . For example, a pipeline could automatically fine-tune the model on issues resolved that day, so it assimilates new bug fixes.

Summary

Our strategy transforms the custom MCP into a full-fledged AI development platform. It unites **Octopus Deploy's AI operations**, **Copilot's customizable agents**, and **DeepMind's advanced reasoning** into a cohesive system. The architecture emphasizes modularity (via MCP and agents), scalability (cloud/on-prem), and continuous improvement (automated pipelines). Agents will think in chains-of-thought (Gemini 2.5) and leverage symbolic reasoning (AlphaGeometry) to provide rigorous analysis, while also iteratively optimizing code (AlphaDev) and generating solutions (AlphaCode). Multi-agent collaboration patterns (parallel brainstorming, sequential pipelines, maker-checker loops) ensure complex tasks are handled robustly ⁹ .

¹⁵ .

By continuously learning from validated sources (via GitHub CLI pipelines ⁷) and testing everything (QA agents and RL feedback), the system self-improves. This AI co-developer will not only suggest code but also validate, optimize, and reason about system design, effectively becoming an intelligent partner in software engineering.

Sources: Octopus AI MCP documentation ¹ ² ; VS Code Copilot MCP docs ⁶ ; GitHub Copilot changelog ³ ⁴ ; VS Code API guides ⁵ ; Octocode (GitHub CLI) analysis ⁷ ; Azure AI agent patterns ¹² ¹³ ¹⁴ ; Anthropic multi-agent research ⁹ ²⁷ ; Gemini 2.5 model card ²⁸ ²⁶ ; AlphaGeometry & AlphaDev research ³² ²⁵ ; AlphaCode study ¹⁶ ; AWS ML pipeline guidance ²³ ; Google Cloud multi-agent reference ⁸ ¹¹ .

¹ ² Use Cases | Documentation and Support

<https://octopus.com/docs/octopus-ai/mcp/use-cases>

³ ⁴ Custom agents for GitHub Copilot - GitHub Changelog

<https://github.blog/changelog/2025-10-28-custom-agents-for-github-copilot/>

⁵ AI extensibility in VS Code | Visual Studio Code Extension API

<https://code.visualstudio.com/api/extension-guides/ai/ai-extensibility-overview>

⁶ Use MCP servers in VS Code

<https://code.visualstudio.com/docs/copilot/customization/mcp-servers>

⁷ Unlocking Deep Code Intelligence: A Hands-On Guide to Octocode MCP Server

<https://skywork.ai/skypage/en/unlocking-deep-code-intelligence-octocode-server/1977929226180300800>

⁸ ¹⁰ ¹¹ ²⁹ ³⁰ Multi-agent AI system in Google Cloud | Cloud Architecture Center

<https://cloud.google.com/architecture/multiagent-ai-system>

⁹ ²¹ ²² ²⁷ ³³ ³⁴ How we built our multi-agent research system \ Anthropic

<https://www.anthropic.com/engineering/multi-agent-research-system>

¹² ¹³ ¹⁴ ¹⁵ ¹⁷ ¹⁸ ¹⁹ AI Agent Orchestration Patterns - Azure Architecture Center | Microsoft Learn

<https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/ai-agent-design-patterns>

¹⁶ Competitive programming with AlphaCode - Google DeepMind

<https://deepmind.google/blog/competitive-programming-with-alphacode/>

²⁰ Agentic AI Architectures And Design Patterns | by Anil Jain | AI / ML Architect | Data Architect | Medium

<https://medium.com/@anil.jain.baba/agentic-ai-architectures-and-design-patterns-288ac589179a>

²³ ML lifecycle architecture diagram - Machine Learning Lens

<https://docs.aws.amazon.com/wellarchitected/latest/machine-learning-lens/ml-lifecycle-architecture-diagram.html>

²⁴ ³² AlphaGeometry: An Olympiad-level AI system for geometry - Google DeepMind

<https://deepmind.google/blog/alphageometry-an-olympiad-level-ai-system-for-geometry/>

²⁵ AlphaDev discovers faster sorting algorithms - Google DeepMind

<https://deepmind.google/blog/alphadev-discovers-faster-sorting-algorithms/>

²⁶ Gemini 2.5: Our newest Gemini model with thinking

<https://blog.google/technology/google-deepmind/gemini-model-thinking-updates-march-2025/>

²⁸ ³¹ Gemini 2.5 Pro - Google DeepMind

<https://deepmind.google/models/gemini/pro/>